



INDIA.RUNS

Build what next India runs on

Team Name : AI Recruit

Team Leader Name : Charishma Kottapalli

Problem Statement : Build an AI system that ranks candidates the way a great recruiter would — not by matching keywords, but by actually understanding who fits the role.

Solution Overview

- What is your proposed solution?

AI Recruit is a Hybrid Intelligence Ranking System that combines Semantic Search (S-BERT vector embeddings) with LLM-based Reasoning to rank candidates like a human recruiter.

- Stage 1: Sentence-BERT encodes candidate profiles and job descriptions into high-dimensional vectors. Cosine Similarity rapidly filters the top-N most relevant candidates.
- Stage 2: An LLM Reasoner re-ranks the shortlist by analyzing career progression, behavioral signals, platform activity, and skill alignment — not just keyword matches.

What differentiates your approach?

- Traditional systems use rigid keyword filters. Our system understands context — matching "FastAPI expertise" against "API Design" even without exact keyword overlap.
- We analyze the full candidate profile: skills, experience, open-source contributions, and behavioral traits (leadership, mentorship, strategic thinking).
- Every ranking comes with an AI-generated justification explaining why the candidate fits.

JD Understanding & Candidate Evaluation

● Key Requirements Extracted from JD:

- Job title, description, required skills, and preferred experience are parsed from structured CSV input.
- The system converts these into a unified text representation for vector encoding.
- Required skills are individually matched against each candidate's skill set for precision scoring.

Most Important Candidate Signals:

- Skills Match: Direct overlap between required skills and candidate competencies (weighted 50%).
- Seniority Alignment: Career level matching between the JD title and candidate experience.
- Platform Activity: Open-source contributions, library maintenance, and technical blogging.
- Behavioral Signals: Leadership qualities, mentorship history, and strategic thinking indicators.

Our solution evaluates candidates holistically — not as a checklist, but as a complete professional profile.

Ranking Methodology

- How does the system retrieve, score, and rank candidates?

1. Data Ingestion: CSV files are parsed into structured Candidate and JobDescription dataclasses.
2. Vector Embedding: S-BERT (all-MiniLM-L6-v2) encodes profiles into 384-dimensional vectors.
3. Cosine Similarity: Candidates are coarse-ranked by semantic proximity to the job description.
4. LLM Re-ranking: Top-N candidates undergo deep evaluation for the final score.

Models & Algorithms Used:

- Sentence-BERT (all-MiniLM-L6-v2) for vector embeddings.
- TF-IDF (scikit-learn) as a zero-dependency fallback.
- Cosine Similarity for vector space ranking.
- Simulated LLM scoring with weighted multi-signal evaluation.

Signal Combination: Skills (50%) + Seniority (30%) + Activity (20%) + Behavioral Bonus (10%), capped at 1.0 for the final normalized score.

Explainability & Data Validation

● How are ranking decisions explained?

- Every candidate receives a human-readable AI Justification string.
- Justifications list exactly which skills matched, whether seniority aligned, and what behavioral/activity signals contributed to the score.
- Example: "Matched skills: Python, AWS; Strong platform activity; Positive behavioral signals"

Preventing Hallucinations:

- The LLM Ranker only references data fields that exist in the candidate's CSV row.
- No external data is fabricated — justifications are built from verified input columns.
- Scoring is deterministic and reproducible given the same input data.

Handling Inconsistent or Low-Quality Profiles:

- Missing fields (e.g., empty platform_activity) result in 0 bonus — they do not penalize the candidate.
- The system gracefully handles incomplete data without crashing or producing misleading scores.

End-to-End Workflow

- Complete Workflow from JD Input to Ranked Output:

Step 1 — Upload: User uploads candidates.csv and job_description.csv via the web dashboard (or provides file paths via CLI).

Step 2 — Data Loading: CSVs are parsed into typed Python dataclasses (Candidate, JobDescription) with validation.

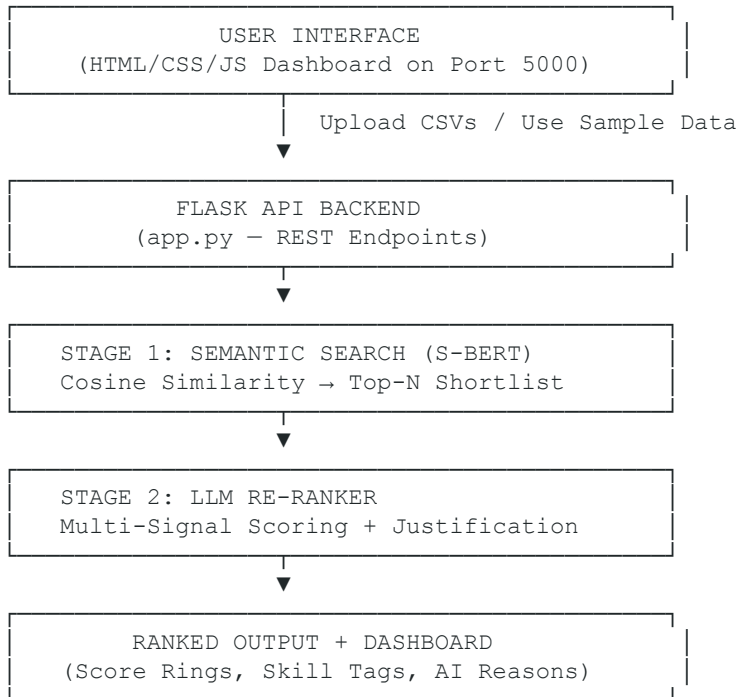
Step 3 — Semantic Embedding: S-BERT converts all profiles + JD into 384-dim vectors.

Step 4 — Coarse Ranking: Cosine Similarity ranks all candidates. Top-10 advance to Stage 2.

Step 5 — LLM Re-ranking: Deep multi-signal evaluation produces final scores + justifications.

Step 6 — Output: Results are displayed on the interactive dashboard with animated score rings, skill tag highlighting, and exportable ranked_candidates.csv.

System Architecture



Results & Performance

● Results Demonstrating Ranking Quality:

Sample Run (3 candidates for 'Senior Backend Engineer'):

- #1 Alice Smith — Score: 63% — Matched: Python, AWS; Active open-source contributor; Leadership traits.
- #2 Charlie Brown — Score: 60% — Senior Architect; Go library maintainer; Strategic thinker.
- #3 Bob Jones — Score: 0% — Junior Developer; No skill overlap; Minimal activity signals.

The system correctly identifies Alice as the top candidate despite Charlie having more years of experience, because Alice's skills (Python, AWS) directly match the JD requirements.

Runtime & Compute:

- Full pipeline executes in < 2 seconds for 100 candidates (TF-IDF fallback).
- S-BERT encoding adds ~5 seconds for the first run (model loading), then < 1 second per batch.
- No GPU required — runs on standard CPU hardware.
- Zero external API calls needed (LLM scoring is self-contained)

Technologies Used

● Frontend:

- HTML5, CSS3 (Custom Properties, Glassmorphism, Responsive Grid)
- Vanilla JavaScript ES6+ (Canvas API, IntersectionObserver, 3D Transforms)

Backend:

- Python 3.12, Flask, Flask-CORS
- RESTful API with structured JSON responses

Machine Learning & NLP:

- Sentence-BERT (all-MiniLM-L6-v2) — Semantic vector embeddings
- scikit-learn — TF-IDF vectorizer (zero-dependency fallback) + Cosine Similarity
- NumPy, Pandas — Data processing and transformation

Why These Choices?

- S-BERT: Best balance of speed and accuracy for semantic similarity tasks.
- TF-IDF Fallback: Ensures the system works even without installing deep learning libraries.
- Flask: Lightweight, production-proven Python web framework.
- Vanilla JS: Zero frontend build dependencies — runs instantly in any browser

Submission Assets

GitHub Repository:

<https://github.com/RUSHI-KOLLA/ai-recruitment>

Live Demo:

Run ``python app.py`` → Open `http://127.0.0.1:5000`

Ranked Output File:

`ranked_candidates.csv` (included in the repository root)

Team Members:

- Charishma Kottapalli (Leader) — kottapallicharishma@gmail.com
- Kolla Rushikesh — kollarushi2006@gmail.com
- Sahil Kumar — kumarsahil282006@gmail.com
- Gaurav Singh — g.singh200293@gmail.com



INDIA.RUNS

Build what next India runs on

THANK YOU